🛒 Online Retail II — Global EDA

Run each cell one by one using Shift+Enter

Cell 1 — Install & Import

```
!pip install openpyxl --quiet

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

# FIX: axes.grid.alpha is not valid in newer matplotlib — removed
plt.rcParams.update({
    "figure.figsize"  : (14, 5),
    "axes.spines.top" : False,
    "axes.spines.right": False,
    "axes.grid"       : True,
    "font.size"       : 11,
})
C = ["#534AB7","#1D9E75","#D85A30","#BA7517","#185FA5","#993C1D"]
print("Setup done")
```

```
Setup done
```

```
FILE = "/content/drive/MyDrive/retail_project/online_retail_II.xlsx"
```

Cell 2 — Load Data

```
from google.colab import drive
drive.mount('/content/drive')

# ⚠️ UPDATE THIS PATH to where your file is in Drive
FILE = "/content/drive/MyDrive/online_retail_II.xlsx"

print("Loading Sheet 1 ... (takes ~30 sec)")
df1 = pd.read_excel(FILE, sheet_name="Year 2009-2010")
print(f"  Sheet 1: {len(df1):,} rows")

print("Loading Sheet 2 ...")
df2 = pd.read_excel(FILE, sheet_name="Year 2010-2011")
print(f"  Sheet 2: {len(df2):,} rows")

df     = pd.concat([df1, df2], ignore_index=True)
df_raw = df.copy()

print(f"\nTotal: {len(df):,} rows  |  {df.shape[1]} columns")
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
Loading Sheet 1 ... (takes ~30 sec)
  Sheet 1: 525,461 rows
Loading Sheet 2 ...
  Sheet 2: 541,910 rows

Total: 1,067,371 rows  |  8 columns
```

Cell 3 — First Look

```
print("Shape:", df.shape)
print("\nColumn types:")
print(df.dtypes)
df.head(8)
```

```
# InvoiceDate = object (text) → fix in cleaning
# Customer ID = float         → ~25% missing (guest buyers)
```

```
Shape: (1067371, 8)

Column types:
Invoice               object
StockCode             object
Description           object
Quantity               int64
InvoiceDate    datetime64[ns]
Price                 float64
Customer ID           float64
Country               object
dtype: object
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085.0	United Kingdom
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom
3	489434	22041	RECORD FRAME 7" SINGLE SIZE	48	2009-12-01 07:45:00	2.10	13085.0	United Kingdom
4	489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	2009-12-01 07:45:00	1.25	13085.0	United Kingdom

Cell 4 — Basic Statistics

```
df.describe().round(2)
# Quantity min < 0  → returns exist
# Price    min = 0  → data errors, must remove
# Cust ID  count < total rows → missing = guest checkouts
```

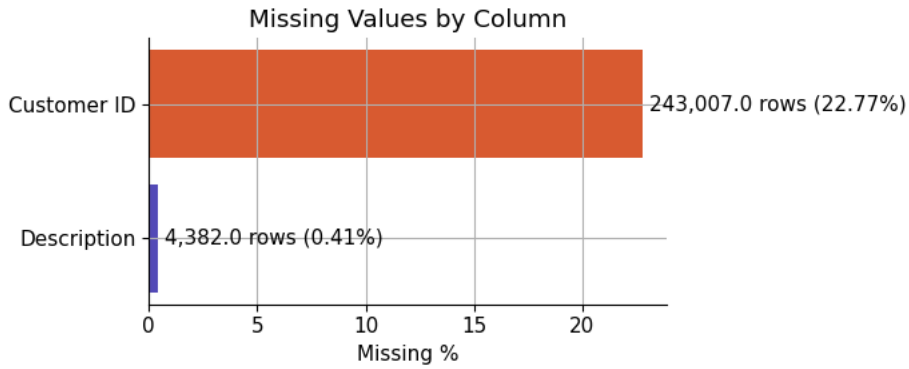
	Quantity	InvoiceDate	Price	Customer ID
count	1067371.00	1067371	1067371.00	824364.00
mean	9.94	2011-01-02 21:13:55.394028544	4.65	15324.64
min	-80995.00	2009-12-01 07:45:00	-53594.36	12346.00
25%	1.00	2010-07-09 09:46:00	1.25	13975.00
50%	3.00	2010-12-07 15:28:00	2.10	15255.00
75%	10.00	2011-07-22 10:23:00	4.15	16797.00
max	80995.00	2011-12-09 12:50:00	38970.00	18287.00
std	172.71	NaN	123.55	1697.46

Cell 5 — Missing Values

```
miss     = df.isnull().sum()
miss_pct = (miss / len(df) * 100).round(2)
miss_df  = pd.DataFrame({"Count": miss, "%": miss_pct})
miss_df  = miss_df[miss_df["Count"] > 0]
print(miss_df)

fig, ax = plt.subplots(figsize=(7, 3))
ax.barh(miss_df.index, miss_df["%"], color=[C[0], C[2]])
for i, (_, row) in enumerate(miss_df.iterrows()):
    ax.text(row["%"] + 0.3, i,
            f"{row['Count']:,} rows ({row['%']}%)", va="center")
ax.set_xlabel("Missing %")
ax.set_title("Missing Values by Column")
plt.tight_layout()
plt.show()

# OBSERVATION:
```

```
# Customer ID ~25% missing → guest checkouts → drop for RFM model
# Description ~0.4% missing → data entry errors → drop
```

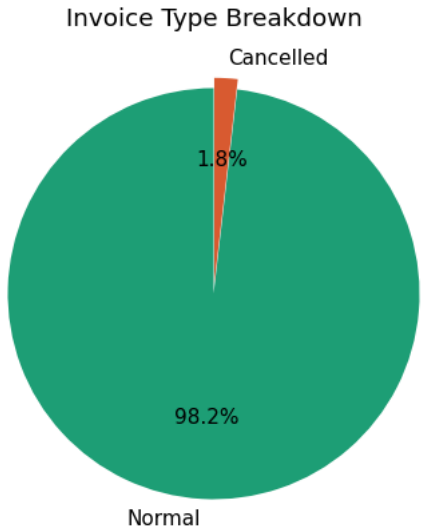
```
              Count      %
Description    4382   0.41
Customer ID  243007  22.77
```



Cell 6 — Duplicates & Invoice Type

```
dups        = df.duplicated().sum()
cancel_mask = df["Invoice"].astype(str).str.startswith("C")

print(f"Duplicate rows    : {dups:,}")
print(f"Cancellations (C.): {cancel_mask.sum():,}  ({cancel_mask.mean()*100:.1f}%)")
print(f"Normal orders     : {(~cancel_mask).sum():,}")

fig, ax = plt.subplots(figsize=(5, 5))
ax.pie([(~cancel_mask).sum(), cancel_mask.sum()],
       labels=["Normal", "Cancelled"],
       autopct="%1.1f%%", colors=[C[1], C[2]],
       startangle=90, explode=[0, 0.05])
ax.set_title("Invoice Type Breakdown")
plt.show()

# OBSERVATION:
# ~2% cancellations → save separately for Anomaly model
```

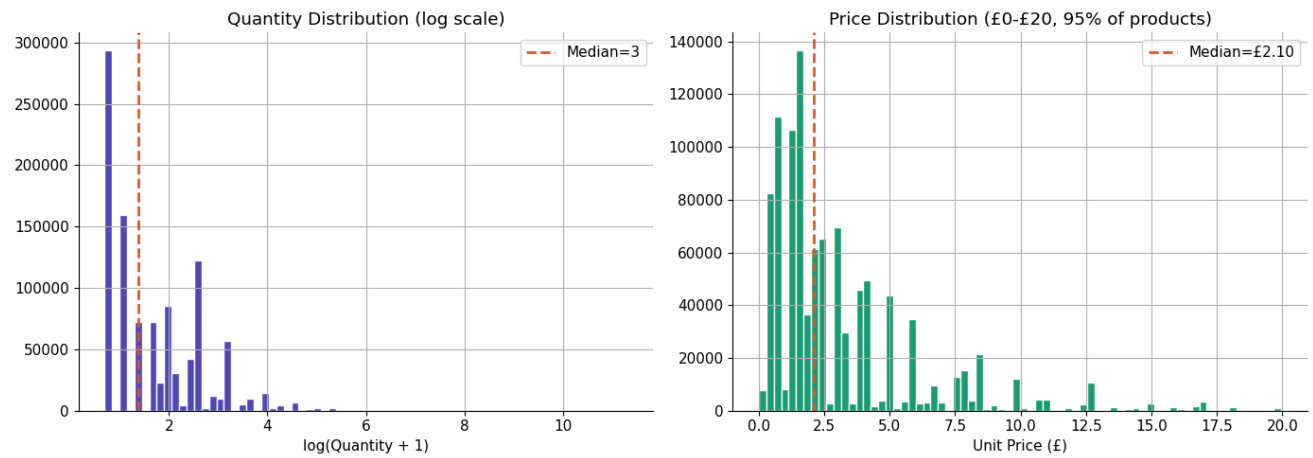
```
Duplicate rows    : 34,335
Cancellations (C.): 19,494  (1.8%)
Normal orders     : 1,047,877
```



Cell 7 — Quantity & Price Distribution

```
valid = df[(df["Quantity"] > 0) & (df["Price"] > 0)]
```

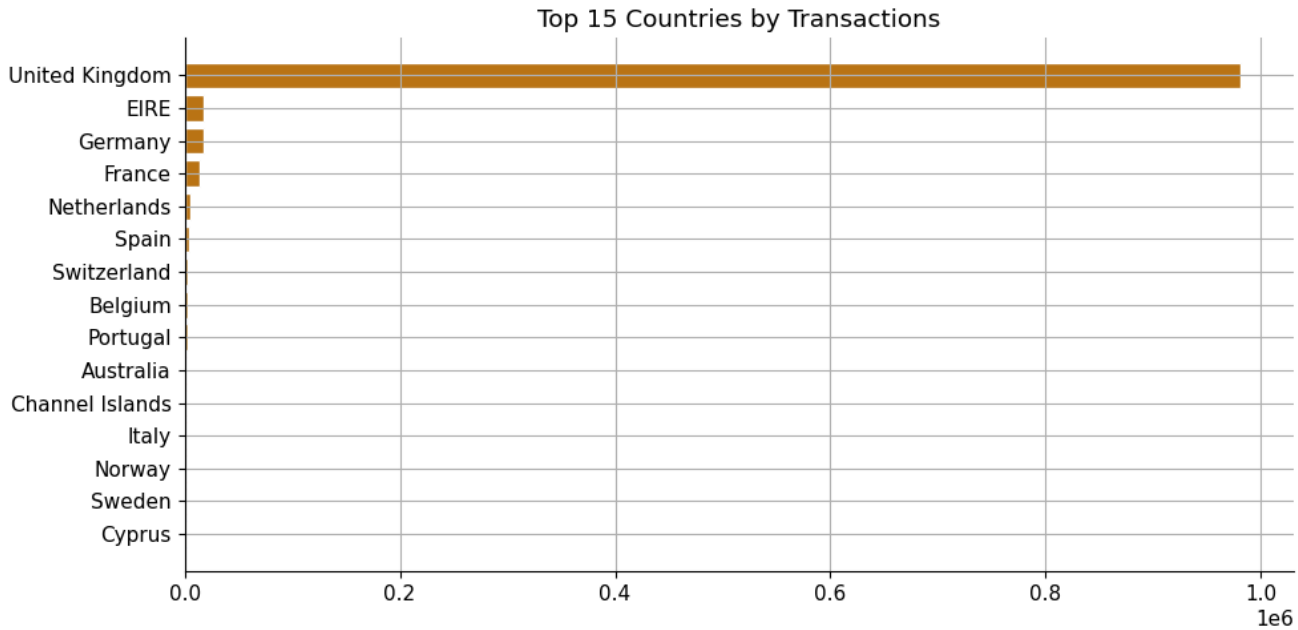
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].hist(np.log1p(valid["Quantity"]), bins=70, color=C[0], edgecolor="white")
axes[0].axvline(np.log1p(valid["Quantity"].median()), color=C[2], lw=2,
                linestyle="--", label=f"Median={valid['Quantity'].median():.0f}")
axes[0].set_title("Quantity Distribution (log scale)")
axes[0].set_xlabel("log(Quantity + 1)")
axes[0].legend()

price_plot = valid[valid["Price"] < 20]["Price"]
axes[1].hist(price_plot, bins=70, color=C[1], edgecolor="white")
axes[1].axvline(price_plot.median(), color=C[2], lw=2,
                linestyle="--", label=f"Median=£{price_plot.median():.2f}")
axes[1].set_title("Price Distribution (£0-£20, 95% of products)")
axes[1].set_xlabel("Unit Price (£)")
axes[1].legend()

plt.tight_layout()
plt.show()

# OBSERVATION:
# Quantity → right-skewed, most orders 1-12 units, few large bulk B2B orders
# Price    → most products £1-£5, confirms cheap gift/home-decor shop
```



Cell 8 — Country Analysis

```
country_counts = df["Country"].value_counts()
uk_pct = country_counts["United Kingdom"] / len(df) * 100
print(f"Unique countries: {df['Country'].nunique()}")
print(f"UK share        : {uk_pct:.1f}%")
print(f"\nTop 10:\n{country_counts.head(10)}")

fig, ax = plt.subplots(figsize=(10, 5))
top15 = country_counts.head(15)
ax.barh(range(15), top15.values[::-1],
        color=[C[0]] + [C[3]]*14, edgecolor="white")
ax.set_yticks(range(15))
ax.set_yticklabels(top15.index[::-1])
ax.set_title("Top 15 Countries by Transactions")
plt.tight_layout()
plt.show()

# OBSERVATION:
# UK = ~90% of transactions → primarily a UK-based B2B retailer
```

```
Unique countries: 43
UK share        : 91.9%

Top 10:
Country
United Kingdom    981330
EIRE               17866
Germany            17624
France             14330
Netherlands         5140
Spain               3811
Switzerland         3189
Belgium             3123
Portugal            2620
Australia           1913
Name: count, dtype: int64
```

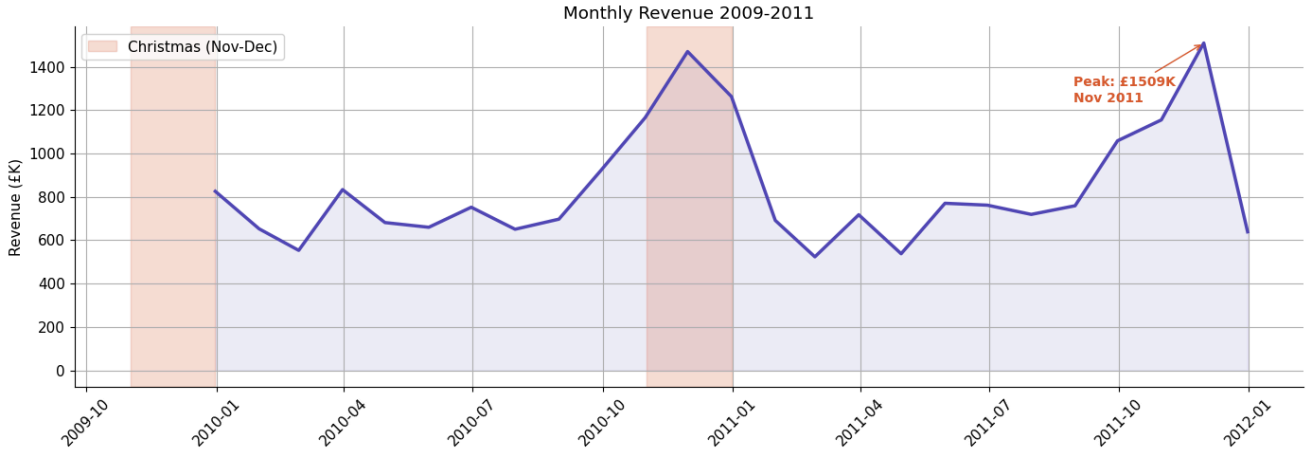


Cell 9 — Monthly Revenue Trend

```
df_t = df[
    (~df["Invoice"].astype(str).str.startswith("C")) &
    (df["Quantity"] > 0) & (df["Price"] > 0)
].copy()
df_t["InvoiceDate"] = pd.to_datetime(df_t["InvoiceDate"])
df_t["Revenue"]     = df_t["Quantity"] * df_t["Price"]

monthly  = df_t.resample("M", on="InvoiceDate")["Revenue"].sum()
xmas_pct = (df_t[df_t["InvoiceDate"].dt.month.isin([11,12])]["Revenue"].sum()
            / df_t["Revenue"].sum() * 100)
print(f"Christmas (Nov+Dec) = {xmas_pct:.1f}% of annual revenue")

fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(monthly.index, monthly/1000, color=C[0], lw=2.5)
ax.fill_between(monthly.index, monthly/1000, alpha=0.1, color=C[0])
for yr in [2009, 2010]:
    ax.axvspan(pd.Timestamp(f"{yr}-11-01"), pd.Timestamp(f"{yr}-12-31"),
               alpha=0.2, color=C[2],
               label="Christmas (Nov-Dec)" if yr == 2009 else "")
peak = monthly.idxmax()
ax.annotate(f"Peak: £{monthly.max()/1000:.0f}K\n{peak.strftime('%b %Y')}",
            xy=(peak, monthly.max()/1000),
            xytext=(peak - pd.DateOffset(months=3), monthly.max()/1000 * 0.82),
            fontsize=10, color=C[2], fontweight="bold",
            arrowprops=dict(arrowstyle="->", color=C[2]))
ax.set_title("Monthly Revenue 2009-2011")
ax.set_ylabel("Revenue (£K)")
ax.legend()
ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```

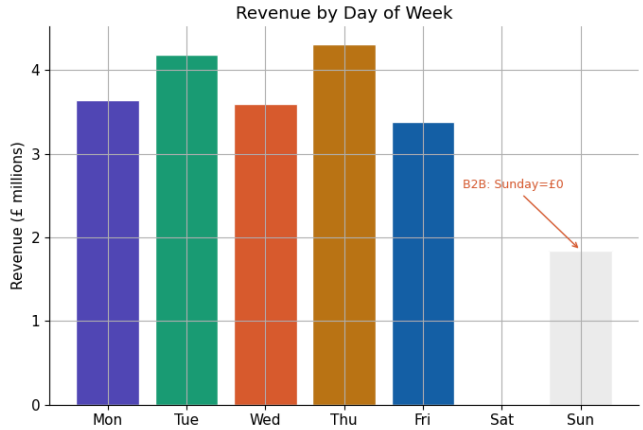
```
# OBSERVATION:
# Strong yearly Christmas spike → plain ARIMA won't capture this
# SARIMA (m=52) or Prophet required for Model 2
```

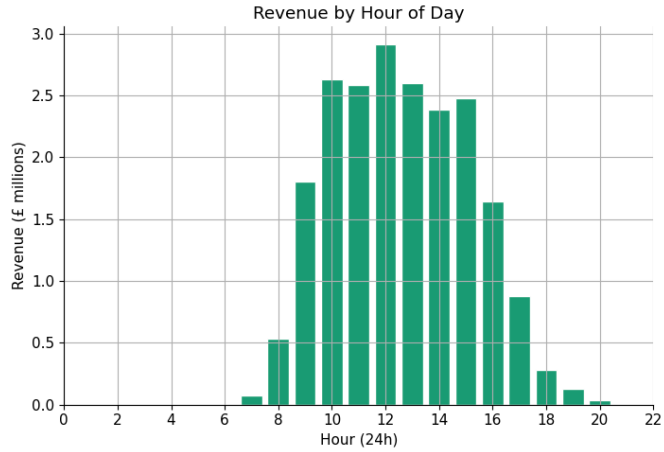
```
Christmas (Nov+Dec) = 27.2% of annual revenue
```



Cell 10 — Day of Week & Hour

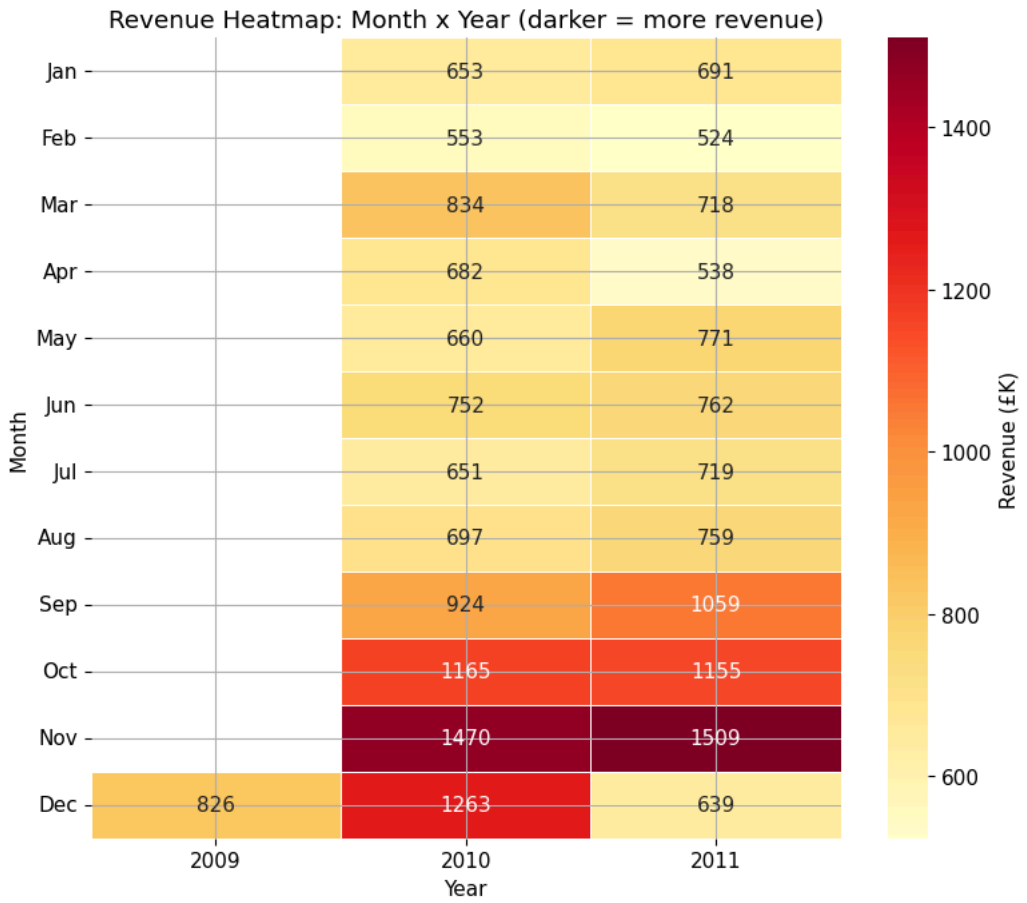
```
df_t["DayName"] = df_t["InvoiceDate"].dt.day_name()
df_t["Hour"]    = df_t["InvoiceDate"].dt.hour

day_order = ["Monday","Tuesday","Wednesday","Thursday","Friday","Saturday","Sunday"]
day_rev   = df_t.groupby("DayName")["Revenue"].sum().reindex(day_order)
hour_rev  = df_t.groupby("Hour")["Revenue"].sum()

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(range(7), day_rev/1e6,
            color=C[:5]+["#ccc","#eee"], edgecolor="white")
axes[0].set_xticks(range(7))
axes[0].set_xticklabels([d[:3] for d in day_order])
axes[0].set_title("Revenue by Day of Week")
axes[0].set_ylabel("Revenue (£ millions)")
axes[0].annotate("B2B: Sunday=£0",
                 xy=(6, day_rev["Sunday"]/1e6),
                 xytext=(4.5, day_rev.max()/1e6 * 0.6),
                 fontsize=9, color=C[2],
                 arrowprops=dict(arrowstyle="->", color=C[2]))

axes[1].bar(hour_rev.index, hour_rev/1e6, color=C[1], edgecolor="white")
axes[1].set_title("Revenue by Hour of Day")
axes[1].set_xlabel("Hour (24h)")
axes[1].set_ylabel("Revenue (£ millions)")
axes[1].set_xticks(range(0, 24, 2))

plt.tight_layout()
plt.show()

# OBSERVATION:
# Sunday = £0, peak 10am-3pm → B2B business, orders during office hours only
```





Cell 11 — Month x Year Heatmap

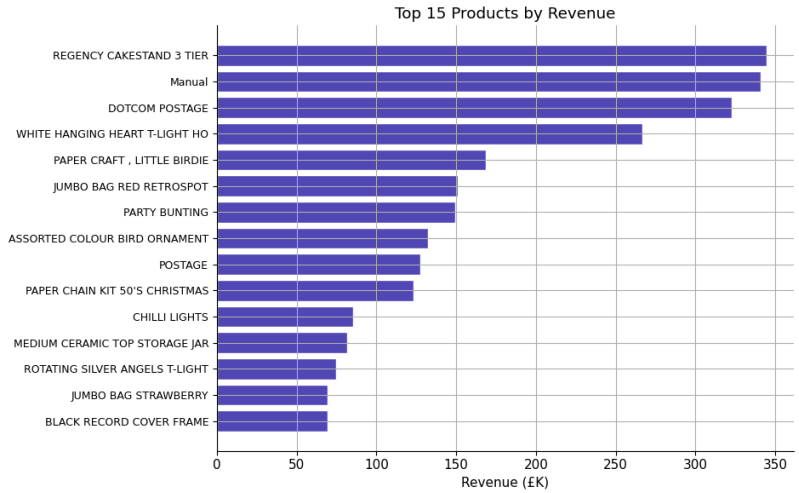
```
df_t["Month"] = df_t["InvoiceDate"].dt.month
df_t["Year"]  = df_t["InvoiceDate"].dt.year

hm = df_t.pivot_table(
    values="Revenue", index="Month", columns="Year", aggfunc="sum"
) / 1000
months = ["Jan","Feb","Mar","Apr","May","Jun",
          "Jul","Aug","Sep","Oct","Nov","Dec"]

fig, ax = plt.subplots(figsize=(8, 7))
sns.heatmap(hm, annot=True, fmt=".0f", cmap="YlOrRd",
            linewidths=0.5, linecolor="white",
            cbar_kws={"label": "Revenue (£K)"}, ax=ax)
ax.set_yticklabels(months, rotation=0)
ax.set_title("Revenue Heatmap: Month x Year (darker = more revenue)")
plt.tight_layout()
plt.show()

# OBSERVATION:
# Nov & Dec darkest every year → Christmas dominates
# Jan & Feb lightest → post-Christmas slump every year
```

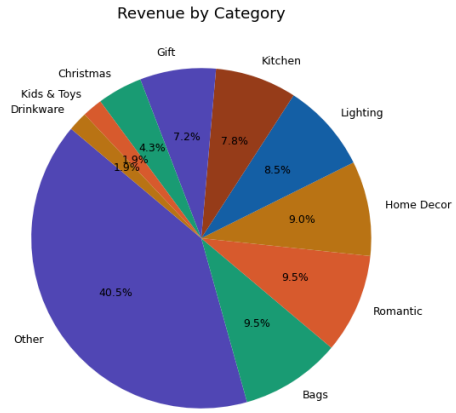


Cell 12 — Top Products & Category

```
def assign_category(d):
    d = str(d).upper()
    if any(w in d for w in ["CHRISTMAS","XMAS","SANTA","SNOWMAN","ADVENT"]): return "Christmas"
    elif any(w in d for w in ["BAG","TOTE","PURSE","POUCH"]):                return "Bags"
    elif any(w in d for w in ["MUG","CUP","BOWL","PLATE","TEA","COFFEE"]):   return "Kitchen"
    elif any(w in d for w in ["CANDLE","LIGHT","LAMP","LANTERN","T-LIGHT"]): return "Lighting"
    elif any(w in d for w in ["FRAME","CLOCK","MIRROR","SIGN","ORNAMENT"]):  return "Home Decor"
    elif any(w in d for w in ["HEART","LOVE","ROSE","FLOWER","FLORAL"]):     return "Romantic"
    elif any(w in d for w in ["TOY","DOLL","GAME","KIDS","CHARLIE"]):        return "Kids & Toys"
    elif any(w in d for w in ["WRAP","GIFT","RIBBON","TAG","CARD"]):         return "Gift"
    elif any(w in d for w in ["BOTTLE","HOT WATER","FLASK"]):                return "Drinkware"
    else:                                                                     return "Other"

df_t["Category"] = df_t["Description"].apply(assign_category)
cat_rev = df_t.groupby("Category")["Revenue"].sum().sort_values(ascending=False)

top15_prod = (df_t.groupby("Description")["Revenue"].sum()
              .sort_values(ascending=False).head(15).reset_index())
top15_prod["Short"] = top15_prod["Description"].str[:30]

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
axes[0].barh(range(15), top15_prod["Revenue"].values[::-1]/1000,
             color=C[0], edgecolor="white")
axes[0].set_yticks(range(15))
axes[0].set_yticklabels(top15_prod["Short"].values[::-1], fontsize=9)
axes[0].set_title("Top 15 Products by Revenue")
axes[0].set_xlabel("Revenue (£K)")

axes[1].pie(cat_rev.values, labels=cat_rev.index,
            autopct="%1.1f%%", colors=C[:len(cat_rev)],
            startangle=140, textprops={"fontsize": 9})
axes[1].set_title("Revenue by Category")
plt.tight_layout()
plt.show()
```

```
# OBSERVATION:
# Christmas items are the top category → confirms strong seasonality in product mix
```

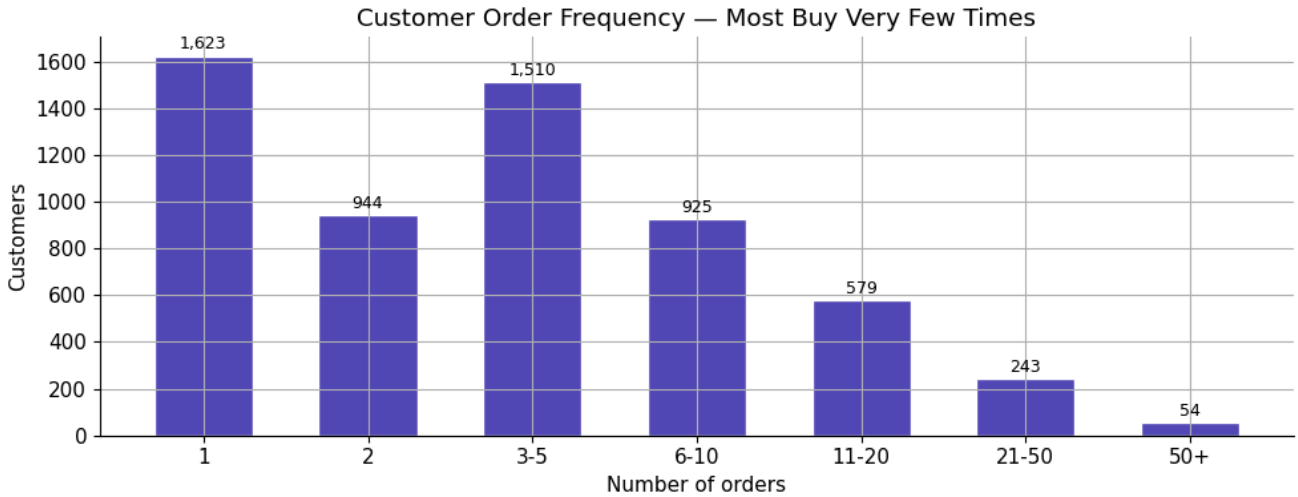




Cell 13 — Customer Behavior

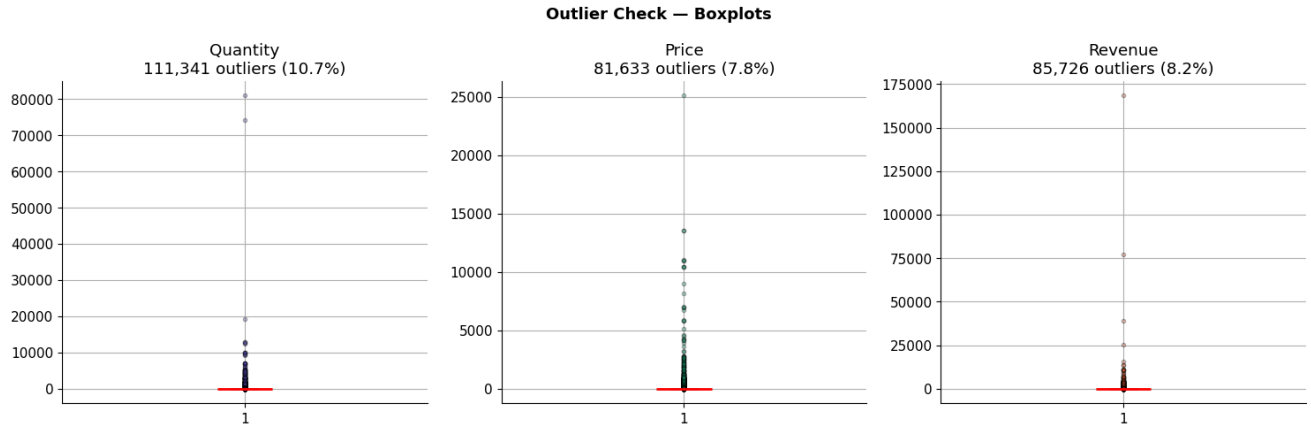
```
orders_per_cust = df_t.groupby("Customer ID")["Invoice"].nunique()
single = (orders_per_cust == 1).sum()

print(f"Unique customers : {orders_per_cust.count():,}")
print(f"Bought only once : {single:,} ({single/len(orders_per_cust)*100:.1f}%) — churn risk")
print(f"Bought 10+ times : {(orders_per_cust >= 10).sum():,} — loyal customers")

freq_labels = ["1","2","3-5","6-10","11-20","21-50","50+"]
freq_bins   = [0,1,2,5,10,20,50,orders_per_cust.max()+1]
freq_dist   = pd.cut(orders_per_cust, bins=freq_bins,
                     labels=freq_labels, right=True).value_counts().reindex(freq_labels)

fig, ax = plt.subplots(figsize=(10, 4))
bars = ax.bar(range(7), freq_dist.values, color=C[0], edgecolor="white", width=0.6)
ax.set_xticks(range(7))
ax.set_xticklabels(freq_labels)
ax.set_title("Customer Order Frequency — Most Buy Very Few Times")
ax.set_xlabel("Number of orders")
ax.set_ylabel("Customers")
for bar, val in zip(bars, freq_dist.values):
    ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+30,
            f"{val:,}", ha="center", fontsize=9)
plt.tight_layout()
plt.show()

# OBSERVATION:
# ~35% one-time buyers → churn is a major problem → motivates Model 3 strongly
```

```
Unique customers : 5,878
Bought only once : 1,623 (27.6%) — churn risk
Bought 10+ times : 974 — loyal customers
```



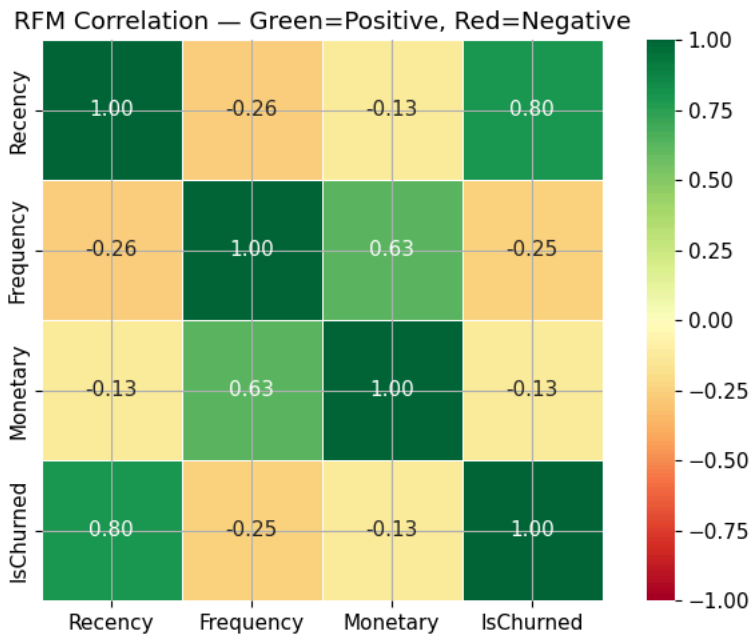
Cell 14 — Outlier Detection

```
valid_rows = df_t[df_t["Quantity"] > 0].copy()

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle("Outlier Check — Boxplots", fontsize=13, fontweight="bold")

for i, (col, color) in enumerate(zip(["Quantity","Price","Revenue"], C[:3])):
    data  = valid_rows[col]
    Q1,Q3 = data.quantile(0.25), data.quantile(0.75)
    IQR   = Q3 - Q1
    n_out = ((data > Q3+1.5*IQR) | (data < Q1-1.5*IQR)).sum()
    axes[i].boxplot(data, patch_artist=True,
                    boxprops=dict(facecolor=color, alpha=0.3),
                    medianprops=dict(color="red", lw=2),
                    flierprops=dict(marker="o", markerfacecolor=color,
                                    markersize=3, alpha=0.3))
    axes[i].set_title(f"{col}\n{n_out:,} outliers ({n_out/len(data)*100:.1f}%)")

plt.tight_layout()
plt.show()

# OBSERVATION:
# All columns have high-end outliers (bulk B2B orders — mostly real data)
# Will use RobustScaler in K-Means (handles outliers better than StandardScaler)
```



Cell 15 — RFM Correlation Check

```
snapshot = df_t["InvoiceDate"].max() + pd.Timedelta(days=1)
quick_rfm = df_t.groupby("Customer ID").agg(
    Recency   = ("InvoiceDate", lambda x: (snapshot - x.max()).days),
    Frequency = ("Invoice",     "nunique"),
    Monetary  = ("Revenue",     "sum"),
).reset_index()
quick_rfm["IsChurned"] = (quick_rfm["Recency"] > 90).astype(int)

corr = quick_rfm[["Recency","Frequency","Monetary","IsChurned"]].corr().round(2)

fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdYlGn",
            center=0, vmin=-1, vmax=1,
            linewidths=0.5, linecolor="white", square=True, ax=ax)
ax.set_title("RFM Correlation — Green=Positive, Red=Negative")
plt.tight_layout()
plt.show()

print(f"Recency vs IsChurned  : {corr.loc['Recency','IsChurned']:+.2f}  — strongest churn signal")
print(f"Frequency vs Monetary : {corr.loc['Frequency','Monetary']:+.2f}  — loyal = big spender")

# OBSERVATION:
# High Recency → likely churned (longer gap = leaving customer)
# All 3 RFM features carry different info → use all 3 together in K-Means
```




```
Recency vs IsChurned  : +0.80  — strongest churn signal
Frequency vs Monetary : +0.63  — loyal = big spender
```

Cell 16 — Data Cleaning

```
print(f"Start: {len(df):,} rows\n")
df_clean = df.copy()

cancel_mask = df_clean["Invoice"].astype(str).str.startswith("C")
df_returns  = df_clean[cancel_mask].copy()
df_clean    = df_clean[~cancel_mask]
print(f"1. Returns separated  : -{cancel_mask.sum():,}  | left: {len(df_clean):,}")

before = len(df_clean)
df_clean = df_clean.dropna(subset=["Customer ID"])
print(f"2. Missing CustID     : -{before-len(df_clean):,}  | left: {len(df_clean):,}")

before = len(df_clean)
df_clean = df_clean.dropna(subset=["Description"])
print(f"3. Missing Desc       : -{before-len(df_clean):,}  | left: {len(df_clean):,}")
```

```
before = len(df_clean)
df_clean = df_clean[df_clean["Quantity"] > 0]
print(f"4. Qty <= 0           : -{before-len(df_clean):,}  | left: {len(df_clean):,}")

before = len(df_clean)
df_clean = df_clean[df_clean["Price"] > 0]
print(f"5. Price <= 0         : -{before-len(df_clean):,}  | left: {len(df_clean):,}")

NON_PROD = ["POST","D","M","DOT","CRUK","BANK CHARGES","AMAZONFEE","PADS","S","C2"]
before = len(df_clean)
df_clean = df_clean[~df_clean["StockCode"].astype(str).str.upper().isin(NON_PROD)]
print(f"6. Non-products       : -{before-len(df_clean):,}  | left: {len(df_clean):,}")

before = len(df_clean)
df_clean = df_clean[df_clean["Quantity"] <= 10000]
print(f"7. Extreme qty >10K   : -{before-len(df_clean):,}  | left: {len(df_clean):,}")

before = len(df_clean)
df_clean = df_clean.drop_duplicates()
print(f"8. Duplicates         : -{before-len(df_clean):,}  | left: {len(df_clean):,}")

df_clean["InvoiceDate"] = pd.to_datetime(df_clean["InvoiceDate"])
df_clean["Customer ID"] = df_clean["Customer ID"].astype(int).astype(str)
df_clean["StockCode"]   = df_clean["StockCode"].astype(str).str.strip().str.upper()
df_clean["Description"] = df_clean["Description"].astype(str).str.strip().str.upper()

print(f"\n{'='*40}")
print(f"CLEAN ROWS   : {len(df_clean):,}")
print(f"RETURNS SAVED: {len(df_returns):,}  (for Anomaly Model 5)")
print(f"{'='*40}")
```

```
Start: 1,067,371 rows

1. Returns separated  : -19,494  | left: 1,047,877
2. Missing CustID     : -242,257  | left: 805,620
3. Missing Desc       : -0  | left: 805,620
4. Qty <= 0           : -0  | left: 805,620
5. Price <= 0         : -71  | left: 805,549
6. Non-products       : -2,870  | left: 802,679
7. Extreme qty >10K   : -7  | left: 802,672
8. Duplicates         : -26,055  | left: 776,617

========================================
CLEAN ROWS   : 776,617
RETURNS SAVED: 19,494  (for Anomaly Model 5)
========================================
```

Cell 17 — Feature Engineering

```
df_clean["Revenue"]     = df_clean["Quantity"] * df_clean["Price"]
df_clean["Year"]        = df_clean["InvoiceDate"].dt.year
df_clean["Month"]       = df_clean["InvoiceDate"].dt.month
df_clean["Week"]        = df_clean["InvoiceDate"].dt.isocalendar().week.astype(int)
df_clean["Quarter"]     = df_clean["InvoiceDate"].dt.quarter
df_clean["DayOfWeek"]   = df_clean["InvoiceDate"].dt.dayofweek
df_clean["Hour"]        = df_clean["InvoiceDate"].dt.hour
df_clean["IsWeekend"]   = (df_clean["DayOfWeek"] >= 5).astype(int)
df_clean["IsChristmas"] = df_clean["Month"].isin([11, 12]).astype(int)
df_clean["Category"]    = df_clean["Description"].apply(assign_category)

print(f"New columns added: Revenue, Year, Month, Week, Quarter,")
print(f"                   DayOfWeek, Hour, IsWeekend, IsChristmas, Category")
print(f"df_clean now has {df_clean.shape[1]} columns")
df_clean.head(3)
```

```
New columns added: Revenue, Year, Month, Week, Quarter,
                   DayOfWeek, Hour, IsWeekend, IsChristmas, Category
df_clean now has 18 columns
```

	Invoice	StockCode	Description	Quantity	InvoiceDate	Price	Customer ID	Country	Revenue	Year	Month	Week	Quarter	DayOfWe
0	489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	2009-12-01 07:45:00	6.95	13085	United Kingdom	83.4	2009	12	49	4	
1	489434	79323P	PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085	United Kingdom	81.0	2009	12	49	4	
2	489434	79323W	WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085	United Kingdom	81.0	2009	12	49	4	

Cell 18 — Build RFM Table

```
snapshot = df_clean["InvoiceDate"].max() + pd.Timedelta(days=1)
print(f"Snapshot date: {snapshot.date()}")

rfm = df_clean.groupby("Customer ID").agg(
    Recency        = ("InvoiceDate",  lambda x: (snapshot - x.max()).days),
    Frequency      = ("Invoice",      "nunique"),
    Monetary       = ("Revenue",      "sum"),
    UniqueProducts = ("StockCode",    "nunique"),
    AvgOrderValue  = ("Revenue",      "mean"),
    TotalItems     = ("Quantity",     "sum"),
    FavCategory    = ("Category",     lambda x: x.mode()[0]),
    Country        = ("Country",      lambda x: x.mode()[0]),
    FirstPurchase  = ("InvoiceDate",  "min"),
    LastPurchase   = ("InvoiceDate",  "max"),
).reset_index()

rfm["CustomerLifetimeDays"] = (rfm["LastPurchase"] - rfm["FirstPurchase"]).dt.days
rfm["AvgDaysBetweenOrders"] = rfm.apply(
    lambda r: round(r["CustomerLifetimeDays"] / r["Frequency"], 1)
    if r["Frequency"] > 1 else 0, axis=1)
rfm["IsChurned"] = (rfm["Recency"] > 90).astype(int)

print(f"RFM table: {len(rfm):,} customers x {rfm.shape[1]} columns")
print(f"Churned (>90d): {rfm['IsChurned'].sum():,} ({rfm['IsChurned'].mean()*100:.1f}%)")
print(f"Active        : {(rfm['IsChurned']==0).sum():,}")
rfm[["Customer ID","Recency","Frequency","Monetary","IsChurned"]].head(5)
```

```
Snapshot date: 2011-12-10
RFM table: 5,861 customers x 14 columns
Churned (>90d): 2,977 (50.8%)
Active        : 2,884
```

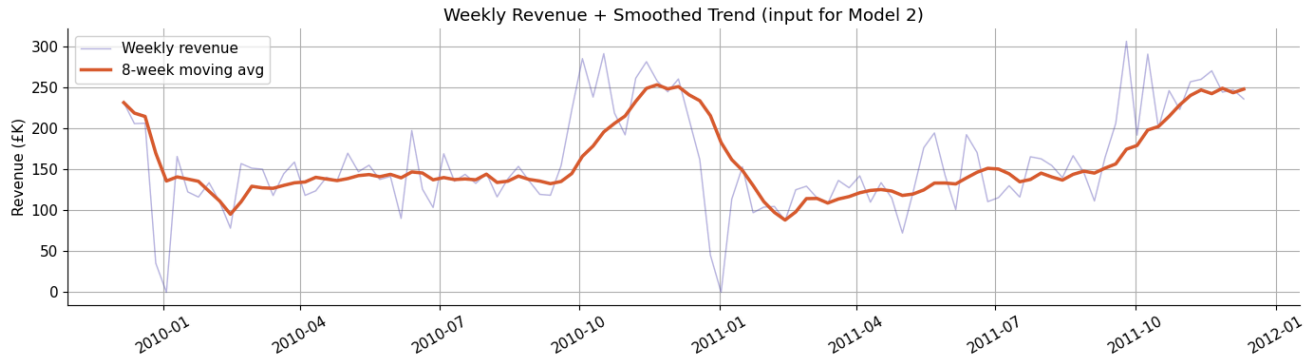
	Customer ID	Recency	Frequency	Monetary	IsChurned
0	12346	529	11	372.86	1
1	12347	2	8	4921.53	0
2	12348	75	5	1658.40	0
3	12349	19	3	3678.69	0
4	12350	310	1	294.40	1

Cell 19 — Build Weekly Time Series

```
ts_weekly = (
    df_clean.resample("W", on="InvoiceDate")["Revenue"]
    .sum().reset_index()
    .rename(columns={"InvoiceDate": "ds", "Revenue": "y"})
)
ts_weekly["y"]            = ts_weekly["y"].clip(lower=0)
```

```
ts_weekly["rolling_4w"]   = ts_weekly["y"].rolling(4, min_periods=1).mean()
ts_weekly["rolling_8w"]   = ts_weekly["y"].rolling(8, min_periods=1).mean()
ts_weekly["month"]        = ts_weekly["ds"].dt.month
ts_weekly["is_christmas"] = ts_weekly["month"].isin([11,12]).astype(int)

print(f"Weekly time series: {len(ts_weekly)} weeks")
print(f"Avg revenue/week  : £{ts_weekly['y'].mean():,.0f}")
print(f"Max revenue/week  : £{ts_weekly['y'].max():,.0f}  (Christmas spike)")

fig, ax = plt.subplots(figsize=(14, 4))
ax.plot(ts_weekly["ds"], ts_weekly["y"]/1000,
        alpha=0.4, color=C[0], lw=1, label="Weekly revenue")
ax.plot(ts_weekly["ds"], ts_weekly["rolling_8w"]/1000,
        color=C[2], lw=2.5, label="8-week moving avg")
ax.set_title("Weekly Revenue + Smoothed Trend (input for Model 2)")
ax.set_ylabel("Revenue (£K)")
ax.legend()
ax.tick_params(axis="x", rotation=30)
plt.tight_layout()
plt.show()
```

```
Weekly time series: 106 weeks
Avg revenue/week  : £158,683
Max revenue/week  : £305,842  (Christmas spike)
```



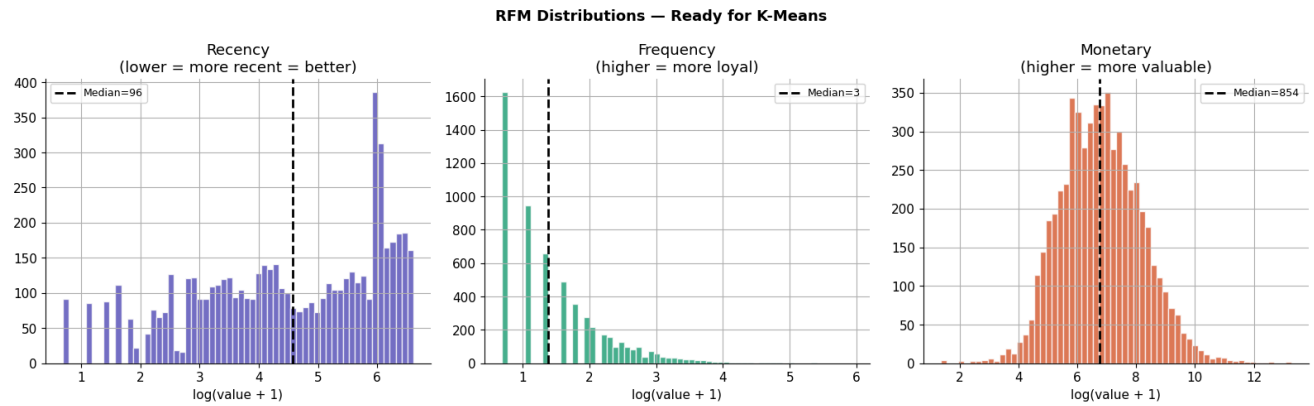
Cell 20 — Add Return Rate to RFM

```
# FIX: check columns carefully before merge to avoid KeyError
if len(df_returns) > 0 and "Customer ID" in df_returns.columns:

    ret = df_returns.dropna(subset=["Customer ID"]).copy()
    ret["Customer ID"] = ret["Customer ID"].astype(float).astype(int).astype(str)

    # count returns per customer
    ret_counts = (
        ret.groupby("Customer ID")["Invoice"]
        .count()
        .reset_index()
    )
    ret_counts.columns = ["Customer ID", "ReturnCount"]  # rename explicitly

    rfm = rfm.merge(ret_counts, on="Customer ID", how="left")
    rfm["ReturnCount"] = rfm["ReturnCount"].fillna(0).astype(int)
    rfm["ReturnRate"]  = (rfm["ReturnCount"] / rfm["Frequency"]).round(4)

else:
    rfm["ReturnCount"] = 0
    rfm["ReturnRate"]  = 0.0

print(f"Return rate added successfully")
print(f"Customers with any returns : {(rfm['ReturnCount'] > 0).sum():,}")
print(f"Max return rate            : {rfm['ReturnRate'].max():.2f}")
print(f"RFM final shape            : {rfm.shape}")
```

```
Return rate added successfully
Customers with any returns : 2,497
Max return rate            : 101.00
RFM final shape            : (5861, 16)
```

Cell 21 — Final RFM Visual Check

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
fig.suptitle("RFM Distributions — Ready for K-Means", fontsize=13, fontweight="bold")

for i, (col, color, note) in enumerate([
    ("Recency",   C[0], "lower = more recent = better"),
    ("Frequency", C[1], "higher = more loyal"),
    ("Monetary",  C[2], "higher = more valuable"),
]):
    axes[i].hist(np.log1p(rfm[col]), bins=60, color=color, edgecolor="white", alpha=0.8)
    axes[i].axvline(np.log1p(rfm[col].median()), color="black", lw=2,
                    linestyle="--", label=f"Median={rfm[col].median():.0f}")
    axes[i].set_title(f"{col}\n({note})")
    axes[i].set_xlabel("log(value + 1)")
    axes[i].legend(fontsize=9)

plt.tight_layout()
plt.show()

# OBSERVATION:
# All 3 features are right-skewed → MUST scale before K-Means
# StandardScaler makes all features equally weighted → fair clustering
```



Cell 22 — Save All Files to Google Drive

```
import os
SAVE = "/content/drive/MyDrive/retail_project/"
os.makedirs(SAVE, exist_ok=True)

df_clean.to_csv(  SAVE + "data_clean.csv",      index=False)
rfm.to_csv(       SAVE + "data_rfm.csv",         index=False)
ts_weekly.to_csv( SAVE + "data_ts_weekly.csv",   index=False)
df_returns.to_csv(SAVE + "data_returns.csv",     index=False)

print("Saved to Google Drive:")
print(f"  data_clean.csv       {len(df_clean):>8,} rows  -> Models 4, 6")
print(f"  data_rfm.csv         {len(rfm):>8,} rows  -> Models 1, 3")
print(f"  data_ts_weekly.csv   {len(ts_weekly):>8}  rows  -> Model 2")
print(f"  data_returns.csv     {len(df_returns):>8,} rows  -> Model 5")
print("\nAll done! Next: Model 1 - K-Means Customer Segmentation")
```

```
Saved to Google Drive:
  data_clean.csv        776,617 rows  -> Models 4, 6
  data_rfm.csv            5,861 rows  -> Models 1, 3
```

```
  data_ts_weekly.csv        106  rows  -> Model 2
  data_returns.csv       19,494 rows  -> Model 5

All done! Next: Model 1 - K-Means Customer Segmentation
```

Cell 23 — EDA Summary

```
print('''
+-------------------------------------------------+
|          GLOBAL EDA — KEY FINDINGS              |
+-------------------------------------------------+
|  ~1M raw rows -> ~750K after cleaning            |
```